

Лабораторная работа №1

Rust: первые программы, функции, структуры и перечисления

Аннотация

Дисциплина: программирование на языке Rust

Раздел курса: базовый синтаксис, ввод данных, функции и пользовательские типы

Формат выполнения: индивидуальная лабораторная работа по одному методическому документу

Среды выполнения: Windows + VS Code + rustup, WSL2 (Ubuntu 24.04+), Linux

Лабораторная работа открывает практическую часть курса по Rust. Студент создаёт два обязательных Cargo-проекта, последовательно осваивает ввод данных, функции, структуры, перечисления, методы, вектор Vec, запуск программ и проверку кода через встроенные тесты.

Оглавление

1	Цель работы	2
2	Правила	2
3	Подготовка рабочей среды	2
3.1	Windows + VS Code + rustup	2
3.2	WSL2 (Ubuntu 24.04+)	2
3.3	Linux	3
4	Как создавать и редактировать файлы	3
5	Создание рабочей структуры лабораторной работы	3
6	Формат сдачи	4
7	Задача 1.1. Карточка студента	4
7.1	Назначение задачи	4
7.2	Шаг 1. Создайте проект	4
7.3	Шаг 2. Запишите файл Cargo.toml	4
7.4	Шаг 3. Запишите файл src/main.rs	4
7.5	Шаг 4. Отформатируйте и проверьте проект	6
7.6	Шаг 5. Запустите программу	6
7.7	Ожидаемый результат	6
7.8	Самопроверка	6
7.9	Типичные ошибки	6
8	Задача 1.2. Мини-каталог книг	6
8.1	Назначение задачи	6
8.2	Шаг 1. Создайте проект	7
8.3	Шаг 2. Запишите файл Cargo.toml	7
8.4	Шаг 3. Запишите файл src/main.rs	7
8.5	Шаг 4. Отформатируйте и проверьте проект	9
8.6	Шаг 5. Запустите программу	9
8.7	Ожидаемый результат	9
8.8	Самопроверка	9
8.9	Типичные ошибки	9
9	Мини-шпаргалка по Cargo	10

1 Цель работы

Цель работы состоит в формировании начального практического навыка разработки небольших консольных программ на Rust с использованием Cargo.

После выполнения работы студент должен уметь:

- создавать бинарный проект командой `cargo new --bin --vcs none`;
- понимать назначение файлов `Cargo.toml` и `src/main.rs`;
- запускать программу командой `cargo run`;
- форматировать код командой `cargo fmt`;
- запускать встроенные тесты командой `cargo test`;
- читать строку из стандартного ввода;
- преобразовывать строку в число через `parse`;
- объявлять функции, структуры, перечисления и методы;
- хранить несколько значений в `Vec` и обрабатывать их через итераторы.

2 Правила

Работа состоит из двух обязательных задач. Они выполняются строго по порядку: сначала 1.1, затем 1.2. Пропуск любой задачи не допускается, потому что вторая задача опирается на навыки, полученные в первой.

В работе используются только стандартная библиотека Rust и стандартный инструмент сборки Cargo. Внешние библиотеки не подключаются, поэтому раздел `[dependencies]` в каждом файле `Cargo.toml` остаётся пустым.

Имена каталогов, файлов и проектов необходимо сохранять без изменений. Это важно для проверки команд, для запуска тестов и для защиты работы.

3 Подготовка рабочей среды

3.1 Windows + VS Code + rustup

В Windows рекомендуется установить Rust через официальный установщик `rustup` со страницы <https://www.rust-lang.org/tools/install>. Если установщик предлагает добавить компоненты Microsoft C++ Build Tools, согласитесь с установкой: они нужны Rust-компилятору для сборки программ под Windows.

После установки откройте новое окно PowerShell и проверьте инструменты:

```
rustup --version
rustc --version
cargo --version
cargo fmt --version
```

Для редактирования кода установите VS Code с сайта <https://code.visualstudio.com/>. Затем откройте панель расширений через `View -> Extensions` или `Ctrl+Shift+X`, найдите расширение `rust-analyzer` и установите его. Это расширение даёт подсветку, подсказки, переходы по коду и сообщения об ошибках прямо в редакторе.

3.2 WSL2 (Ubuntu 24.04+)

В WSL2 откройте терминал Ubuntu и установите Rust через `rustup`:

```
sudo apt update
sudo apt install -y curl build-essential
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
```

После установки проверьте инструменты:

```
rustup --version
rustc --version
cargo --version
cargo fmt --version
```

Если вы работаете в VS Code из WSL2, установите в VS Code расширение WSL от Microsoft и расширение rust-analyzer. Открывать каталог проекта удобно из терминала Ubuntu командой `code .`; в левом нижнем углу окна VS Code должен быть виден признак WSL-среды.

3.3 Linux

В установленной Linux-системе используйте rustup. Для Debian/Ubuntu предварительно установите базовые инструменты:

```
sudo apt update
sudo apt install -y curl build-essential
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
```

Проверка выполняется так:

```
rustup --version
rustc --version
cargo --version
cargo fmt --version
```

В качестве редактора можно использовать VS Code с расширением rust-analyzer, консольный редактор nano или любой другой текстовый редактор, который сохраняет обычные текстовые файлы в кодировке UTF-8.

4 Как создавать и редактировать файлы

В этой работе каждый проект создаётся командой `cargo new`. Cargo сам создаёт каталог проекта, файл `Cargo.toml` и файл `src/main.rs`. После этого содержимое этих файлов нужно заменить на тексты, приведённые в соответствующей задаче.

Если вы работаете в VS Code, откройте папку лабораторной работы через `File -> Open Folder...`. Файлы удобно создавать и открывать в панели Explorer, которая включается через `View -> Explorer` или `Ctrl+Shift+E`. После вставки текста сохраняйте файл через `File -> Save` или `Ctrl+S`.

Если вы работаете в терминале, можно использовать nano:

```
nano src/main.rs
```

В nano сохранение выполняется клавишами `Ctrl+O`, затем `Enter`, выход — `Ctrl+X`.

5 Создание рабочей структуры лабораторной работы

Создайте общий каталог для Rust-лабораторных работ и перейдите в него.

В Windows PowerShell:

```
mkdir rust-labs
cd rust-labs
mkdir 01-rust-basics
cd 01-rust-basics
```

В WSL2 или Linux:

```
mkdir -p ~/rust-labs/01-rust-basics
cd ~/rust-labs/01-rust-basics
```

После этого выполняйте задачи 1.1 и 1.2 строго в указанном порядке.

6 Формат сдачи

На защите необходимо показать два проекта: `task01_student_card` и `task02_library_catalog`. Для каждого проекта нужно продемонстрировать содержимое `Cargo.toml`, содержимое `src/main.rs`, успешное выполнение `cargo fmt`, успешное выполнение `cargo test` и запуск программы через `cargo run --quiet`.

7 Задача 1.1. Карточка студента

Уровень сложности: базовый, первая обязательная задача.

7.1 Назначение задачи

В этой задаче программа запрашивает у пользователя имя, группу, год рождения и текущий год, после чего выводит аккуратную карточку студента. На небольшом примере осваиваются переменные, строки `String`, чтение из стандартного ввода, преобразование текста в число и выделение повторяемой логики в функции.

7.2 Шаг 1. Создайте проект

Находясь в каталоге `01-rust-basics`, выполните:

```
cargo new --bin --vcs none task01_student_card
cd task01_student_card
```

Команда создаёт бинарный проект. Слово `--bin` означает, что проект будет исполняемой программой, а `--vcs none` запрещает Cargo создавать отдельный Git-репозиторий внутри учебной папки.

7.3 Шаг 2. Запишите файл `Cargo.toml`

Итоговое содержимое файла `Cargo.toml` должно быть следующим:

```
[package]
name = "task01_student_card"
version = "0.1.0"
edition = "2024"
```

```
[dependencies]
```

Файл `Cargo.toml` называется манифестом проекта. В нём указаны имя проекта, версия, редакция языка Rust и список зависимостей. В этой работе внешние зависимости не нужны, поэтому раздел `[dependencies]` остаётся пустым.

7.4 Шаг 3. Запишите файл `src/main.rs`

Итоговое содержимое файла `src/main.rs` должно быть следующим:

```
use std::io;

fn read_trimmed_line(prompt: &str) -> String {
    println!("{}", prompt);

    let mut input = String::new();
    io::stdin()
        .read_line(&mut input)
        .expect("failed to read input line");

    input.trim().to_string()
}
```

```

fn parse_year(text: &str) -> u32 {
    text.trim()
        .parse::<u32>()
        .expect("year must be a positive integer")
}

fn calculate_age(birth_year: u32, current_year: u32) -> u32 {
    current_year.saturating_sub(birth_year)
}

fn build_student_card(name: &str, group: &str, birth_year: u32, current_year: u32) -> String {
    let age = calculate_age(birth_year, current_year);

    format!(
        "--- Student card ---\nName: {name}\nGroup: {group}\nBirth year: {birth_year}\nCurrent year: {current_year}\nAge: {age}"
    )
}

fn main() {
    let name = read_trimmed_line("Enter student name:");
    let group = read_trimmed_line("Enter group:");
    let birth_year_text = read_trimmed_line("Enter birth year:");
    let current_year_text = read_trimmed_line("Enter current year:");

    let birth_year = parse_year(&birth_year_text);
    let current_year = parse_year(&current_year_text);

    let card = build_student_card(&name, &group, birth_year, current_year);
    println!("{}", card);
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn calculates_age_for_normal_years() {
        assert_eq!(calculate_age(2006, 2026), 20);
    }

    #[test]
    fn parses_year_with_spaces() {
        assert_eq!(parse_year(" 2026 "), 2026);
    }

    #[test]
    fn builds_card_with_expected_lines() {
        let card = build_student_card("Anna", "RUST-101", 2006, 2026);

        assert!(card.contains("Name: Anna"));
        assert!(card.contains("Group: RUST-101"));
        assert!(card.contains("Age: 20"));
    }
}

```

В программе функция `read_trimmed_line` читает строку и убирает лишние пробелы по краям. Функция `parse_year` преобразует текст в число `u32`. Функция `calculate_age` вычисляет возраст, а `build_student_card` собирает многострочный текст карточки. Благодаря такому разделению код легче читать и проверять тестами.

7.5 Шаг 4. Отформатируйте и проверьте проект

Выполните команды:

```
cargo fmt
cargo test
```

Команда `cargo fmt` приводит код к стандартному стилю Rust. Команда `cargo test` запускает встроенные тесты из блока `#[cfg(test)]`.

7.6 Шаг 5. Запустите программу

В Windows PowerShell можно выполнить:

```
"Anna`nRUST-101`n2006`n2026" | cargo run --quiet
```

В WSL2 или Linux выполните:

```
printf 'Anna\nRUST-101\n2006\n2026\n' | cargo run --quiet
```

Можно также запустить `cargo run --quiet` без конвейера и ввести строки вручную после появления приглашений.

7.7 Ожидаемый результат

```
Enter student name:
Enter group:
Enter birth year:
Enter current year:
--- Student card ---
Name: Anna
Group: RUST-101
Birth year: 2006
Current year: 2026
Age: 20
```

7.8 Самопроверка

Проверьте, что команда `cargo test` завершилась без ошибок, а вывод программы содержит имя, группу и возраст. Если изменить год рождения во входных данных, возраст в последней строке также должен измениться.

7.9 Типичные ошибки

Если Rust сообщает, что не может преобразовать строку в число, проверьте, что в строках года рождения и текущего года введены только цифры. Если команда `cargo run` запускает старую версию программы, сохраните файл `src/main.rs` и повторите запуск. Если VS Code подчёркивает код сразу после вставки, сначала сохраните файл и выполните `cargo fmt`.

8 Задача 1.2. Мини-каталог книг

Уровень сложности: базовый с расширением, вторая обязательная задача.

8.1 Назначение задачи

Во второй задаче программа создаёт небольшой каталог книг и выводит отчёт: список книг, общее количество, количество научных книг, сумму страниц и самую новую книгу. В задаче используются `struct`, `enum`, блоки `impl`, вектор `Vec` и функции, принимающие срез `&[Book]`.

8.2 Шаг 1. Создайте проект

Вернитесь в каталог 01-rust-basics и создайте второй проект:

```
cd ..  
cargo new --bin --vcs none task02_library_catalog  
cd task02_library_catalog
```

Если вы уже находитесь в каталоге 01-rust-basics, первую команду `cd ..` выполнять не нужно.

8.3 Шаг 2. Запишите файл Cargo.toml

Итоговое содержимое файла Cargo.toml должно быть следующим:

```
[package]  
name = "task02_library_catalog"  
version = "0.1.0"  
edition = "2024"  
  
[dependencies]
```

8.4 Шаг 3. Запишите файл src/main.rs

Итоговое содержимое файла src/main.rs должно быть следующим:

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]  
enum Genre {  
    Fiction,  
    Science,  
    History,  
}  
  
impl Genre {  
    fn title(&self) -> &'static str {  
        match self {  
            Genre::Fiction => "Fiction",  
            Genre::Science => "Science",  
            Genre::History => "History",  
        }  
    }  
}  
  
#[derive(Debug)]  
struct Book {  
    title: String,  
    author: String,  
    year: u16,  
    genre: Genre,  
    pages: u16,  
}  
  
impl Book {  
    fn new(title: &str, author: &str, year: u16, genre: Genre, pages: u16) -> Self {  
        Self {  
            title: title.to_string(),  
            author: author.to_string(),  
            year,  
            genre,  
            pages,  
        }  
    }  
  
    fn describe(&self) -> String {
```

```

        format!(
            "{} - {}, {}, {}, {} pages",
            self.title,
            self.author,
            self.year,
            self.genre.title(),
            self.pages
        )
    }
}

fn total_pages(books: &[Book]) -> u32 {
    books.iter().map(|book| u32::from(book.pages)).sum()
}

fn count_by_genre(books: &[Book], genre: Genre) -> usize {
    books.iter().filter(|book| book.genre == genre).count()
}

fn newest_book(books: &[Book]) -> Option<&Book> {
    books.iter().max_by_key(|book| book.year)
}

fn main() {
    let books = vec![
        Book::new("The Rust Journey", "I. Ferris", 2024, Genre::Science, 320),
        Book::new("Safe Systems", "A. Memory", 2022, Genre::Science, 280),
        Book::new("Northern Tales", "L. Snow", 2019, Genre::Fiction, 210),
        Book::new("History of Code", "M. Archive", 2021, Genre::History, 260),
    ];

    println!("Library catalog");
    for book in &books {
        println!("- {}", book.describe());
    }

    println!("Total books: {}", books.len());
    println!("Science books: {}", count_by_genre(&books, Genre::Science));
    println!("Total pages: {}", total_pages(&books));

    if let Some(book) = newest_book(&books) {
        println!("Newest book: {} ({})", book.title, book.year);
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    fn sample_books() -> Vec<Book> {
        vec![
            Book::new("A", "Author 1", 2020, Genre::Science, 100),
            Book::new("B", "Author 2", 2022, Genre::Fiction, 150),
            Book::new("C", "Author 3", 2021, Genre::Science, 200),
        ]
    }

    #[test]
    fn counts_books_by_genre() {
        let books = sample_books();
    }
}

```



```

        assert_eq!(count_by_genre(&books, Genre::Science), 2);
        assert_eq!(count_by_genre(&books, Genre::History), 0);
    }

    #[test]
    fn sums_pages() {
        let books = sample_books();

        assert_eq!(total_pages(&books), 450);
    }

    #[test]
    fn finds_newest_book() {
        let books = sample_books();
        let newest = newest_book(&books).expect("sample list is not empty");

        assert_eq!(newest.title, "B");
        assert_eq!(newest.year, 2022);
    }
}

```

Перечисление `Genre` задаёт допустимые жанры книг. Структура `Book` объединяет название, автора, год, жанр и количество страниц. Метод `Book::new` создаёт новую книгу, а метод `describe` возвращает строку для печати. Функции `total_pages`, `count_by_genre` и `newest_book` показывают, как обрабатывать не один объект, а весь список.

8.5 Шаг 4. Отформатируйте и проверьте проект

```

cargo fmt
cargo test

```

8.6 Шаг 5. Запустите программу

Команда одинакова для Windows PowerShell, WSL2 и Linux:

```

cargo run --quiet

```

8.7 Ожидаемый результат

```

Library catalog
- The Rust Journey – I. Ferris, 2024, Science, 320 pages
- Safe Systems – A. Memory, 2022, Science, 280 pages
- Northern Tales – L. Snow, 2019, Fiction, 210 pages
- History of Code – M. Archive, 2021, History, 260 pages
Total books: 4
Science books: 2
Total pages: 1070
Newest book: The Rust Journey (2024)

```

8.8 Самопроверка

Убедитесь, что `cargo test` показывает три успешных теста. Затем сравните вывод `cargo run --quiet` с ожидаемым результатом. Если изменить количество страниц у одной книги, строка `Total pages` должна измениться.

8.9 Типичные ошибки

Если компилятор сообщает, что тип `Genre` нельзя сравнивать, проверьте строку `#[derive(Clone, Copy, Debug, PartialEq, Eq)]`. Если ошибка возникает в `newest_book`, убедитесь, что функция

возвращает `Option<Book>`, потому что список книг теоретически может быть пустым. Если вывод отличается, проверьте порядок книг в векторе `books`.

9 Мини-шпаргалка по Cargo

- `cargo new --bin --vcs none имя_проекта` — создать новый исполняемый проект без отдельного Git-репозитория.
- `cargo fmt` — отформатировать код по стандарту Rust.
- `cargo fmt --check` — проверить форматирование без изменения файлов.
- `cargo test` — запустить встроенные тесты.
- `cargo run` — собрать и запустить программу.
- `cargo run --quiet` — запустить программу без лишних сообщений Cargo.

10 Финальный чек-лист сдачи

1. Создан каталог `01-rust-basics`.
2. Создан проект `task01_student_card`.
3. В проекте `task01_student_card` заполнены `Cargo.toml` и `src/main.rs`.
4. Для `task01_student_card` успешно выполнены `cargo fmt`, `cargo test`, `cargo run --quiet`.
5. Создан проект `task02_library_catalog`.
6. В проекте `task02_library_catalog` заполнены `Cargo.toml` и `src/main.rs`.
7. Для `task02_library_catalog` успешно выполнены `cargo fmt`, `cargo test`, `cargo run --quiet`.
8. Оба проекта продемонстрированы преподавателю в установленном порядке: 1.1 -> 1.2.